

FraudGuard - ML Project Documentation

Project Overview

FraudGuard is a real-time fraud detection system designed for high-throughput financial transaction processing. The system analyzes transaction patterns, user behavior, and contextual features to identify potentially fraudulent activities in real-time with sub-100ms latency requirements.

Business Problem

Financial fraud costs businesses over \$50 billion annually. Traditional rule-based systems generate high false positive rates and miss sophisticated fraud patterns. FraudGuard uses machine learning to reduce false positives while catching more fraudulent transactions through adaptive pattern recognition.

Architecture

The system follows a lambda architecture with both batch and streaming components:

Data Ingestion Layer:

- Apache Kafka for real-time transaction streaming
- AWS S3 for historical data lake storage
- Apache Airflow for batch ETL pipelines

Feature Engineering:

- Redis for real-time feature store (transaction velocity, amount patterns)
- Feature computation pipeline with Apache Flink
- Feature versioning with MLflow

Model Serving:

- FastAPI endpoints behind load balancer
- Model ensemble: XGBoost + Neural Network voting
- A/B testing framework for model comparison

Monitoring:

- Prometheus + Grafana for system metrics
- Custom dashboards for model performance
- Automated alerting for drift detection

Technical Implementation

Feature Engineering:

- Transaction amount z-scores per user (rolling 30-day window)
- Transaction frequency features (hourly, daily, weekly patterns)
- Merchant category embeddings
- Geographic distance from user's typical locations
- Device fingerprint and IP geolocation features
- Time-since-last-transaction features

FraudGuard - ML Project Documentation

Model Architecture:

- Primary: XGBoost with custom objective function (weighted cross-entropy)
- Secondary: 1D-CNN for sequential transaction patterns
- Ensemble: Soft voting with learned weights
- Threshold optimization using cost-sensitive metrics

Training Pipeline:

- Weekly retraining on latest 90 days of data
- SMOTE for handling class imbalance (0.1% fraud rate)
- Hyperparameter tuning with Optuna
- Model validation with stratified time-series split

Results

Performance Metrics:

- Precision: 0.92 (up from 0.78 with rule-based system)
- Recall: 0.95 (up from 0.85)
- F1 Score: 0.935
- AUC-ROC: 0.987
- False Positive Rate: Reduced by 62%

Business Impact:

- Detected \$2.1M in fraudulent transactions in first quarter
- Reduced manual review workload by 45%
- Average detection latency: 47ms
- System uptime: 99.97%

Challenges and Solutions

1. Class Imbalance: Only 0.1% of transactions are fraudulent
Solution: Combined SMOTE oversampling with cost-sensitive learning and custom loss functions
2. Real-time Requirements: Sub-100ms response time needed
Solution: Redis feature store, model optimization with ONNX, batch prediction for non-critical paths
3. Concept Drift: Fraud patterns evolve constantly
Solution: Automated drift detection, weekly retraining, champion-challenger model deployment
4. Interpretability: Regulatory requirements for explainable decisions
Solution: SHAP values for feature importance, natural language explanations for alerts

Lessons Learned

- Start with simple baselines before complex models
- Feature engineering often matters more than model architecture

FraudGuard - ML Project Documentation

- Production ML requires significant infrastructure investment
- Monitoring and observability are as important as model performance
- Cross-functional collaboration with business stakeholders is critical